澄清项

关于

- 1、手动下架的前置条件：是否只需在售即可？下架后队列中已有意向是“作废”还是仅“失效”？是否允许有排队时下架？
- 2、交易失败重排后的口令码：沿用原码还是发新码？沿用的话买家如何知道已重排；作废/重排分别怎么失效？
- 3、手动下架的商品进入历史后，“最终交易结果”怎么标？和“交易成功下架”怎么区分？（记录里明确说这一块是缺口）
- 4、商品图片：卖家发布时是填 `URL` 还是上传本地文件？JPG/PNG、≤5MB 的校验放在前端还是后端？
- 5、重排后队首是否立即自动进入交易并冻结？重排次数要不要上限，避免“失败→重排→再失败”死循环？

  这些问题，的回复：

  对于1：

  答：手动下架我之前提过这个想法——主要是想，现在卖的这个商品如果不想卖了（比如想换个商品），能把它从"在售"撤下来，空出位置发新的。

  不过你问的这几个细节，说实话我还没细想：

  - 前置条件是不是只要"在售"就行——我大概觉得是，商品还在卖才能撤，但要不要再加别的限制我还没定。
  - 下架之后，还在排队的那些人怎么办，是彻底作废还是先挂着以后再说——这块我没想清楚。
  - 有排队的时候允不允许下架——也没想好。

  这些规则要不你帮我一起想想，怎么定比较合理？

  我再问：

  这样怎么样：前置条件：仅「在售」可手动下架。
  交易中/冻结中：不能直接手动下架，必须先「标记交易结果」结束交易；若是「手动冻结」，先解冻再下架。
  有排队时：允许下架。
  下架瞬间：队列中所有未成功意向统一置为终态「失败」，口令码失效；买家再查询提示「口令码已失效」。
  通知：系统无通知机制，因此下架前卖家后台必须弹二次确认，明确提示“将失效 N 个购买意向，且系统不会通知买家”。
  历史：商品进历史，商品级结果标为「手动下架」；该商品下的意向人终态为「失败」。
  后续：下架后可立即发布新商品，旧商品保留在历史中。

  答：

  想得挺细，不过说实话，比我要的复杂多了。

  我提"手动下架"就一个很朴素的诉求：现在卖的商品不想卖了（比如想换个商品），能把它从"在售"撤下来，好空出位置发新的。就这么简单。

  你加的这些规则——前置条件、冻结状态怎么切、意向人统一置"失败"、二次确认弹窗、历史里专门标"手动下架"——都是过度设计了，我原本没往这么细想。

  只有两点我认：下架后买家手上的口令码就失效了（交易结束或商品下架后口令码查不了，这点我之前说过）；下架后我想能马上发新商品。

其他那套复杂规则，我觉得不用搞这么重。你觉得呢？

对于2：

答：交易一结束，口令码就没用了，买家那边查不了也改不了。重排是卖家后台的事：卖家决定这个人还要不要留在队里。留的话，他的意向就还在队里排着，等下轮交易轮到他，卖家直接线下联系他——他填过电话，这电话本来就是为了线下联系用的。

所以不存在"沿用原码还是发新码"：失效就失效了，不补新码，也不给买家开一个查重排状态的口子。

对于3：

答："手动下架"你抠得太深了，我直说：我的需求里，商品下架就只有一种情况——交易成功下架，没有"手动下架"这个独立类型。

"最终交易结果"标的是交易结果，就两种：交易成功、交易失败。它不是用来区分下架方式的。

所以不存在"手动下架进历史怎么标"，也不存在"和交易成功下架区分"——下架就一条路：卖出去、交易成功、进历史。

对于4：

答：图片是上传本地文件，不是填 URL——我发布商品时直接传图片文件上去，一张主图就够了（至多 1 张），JPG 或 PNG 都行，别超过 5MB，超了系统得给我个提示。

至于"JPG/PNG、≤5MB 的校验放前端还是后端"——这是你们技术实现的事，超出需求讨论范围了，我只负责聊产品需求。

对于5：

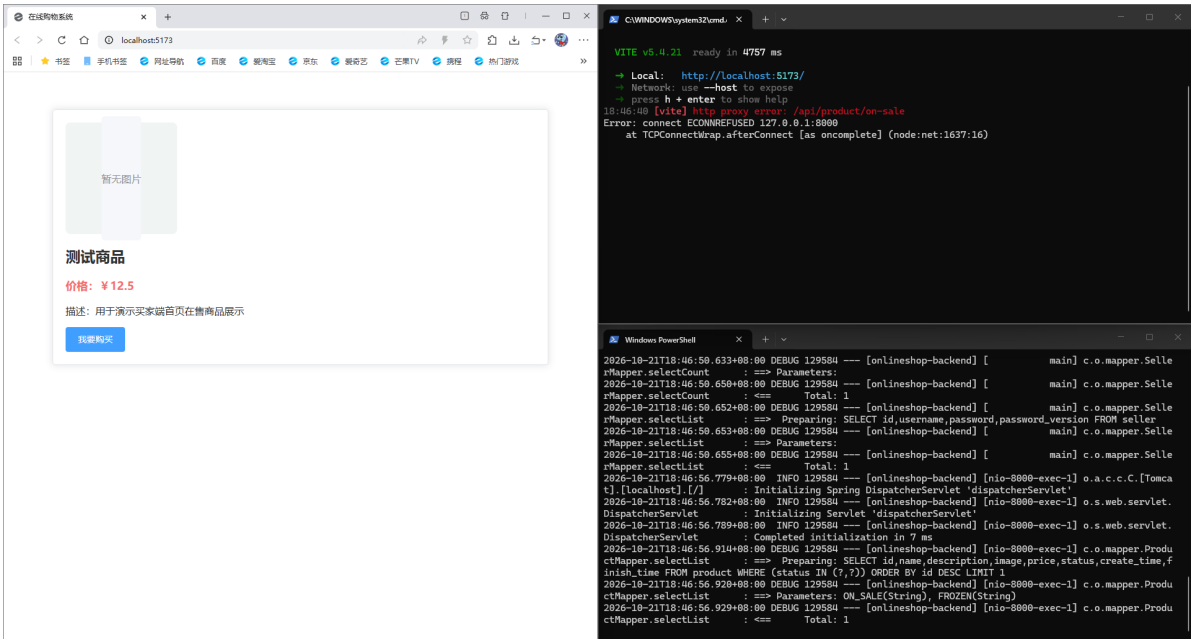
答：递补：交易失败后商品恢复"在售"，排队的人按提交顺序往前挪，新的队首成为下一个交易对象。但"进入交易、冻结"不是立即自动的——得卖家发起（线下联系队首、进入交易流程），那一刻才自动冻结。

重排次数：没有上限。重排是卖家每次交易失败后主动确认的，作废还是重排卖家说了算，想停随时作废就行，所以不存在自动的"失败→重排→再失败"死循环。

基于这些结论，我把受影响的 `US-10`、`US-14`、`US-17`、`US-20`、`US-21`、`US-22`、`US-24`、`US-27`、`US-28` 等用户故事都做了修订，并在项目 wiki 上发布了 `需求池-v2.md`。

买家端负责四个页面：商品首页、购买表单、结果页、口令查询页，技术栈是 Vue3 + Vite + Element Plus。

1.首页在售商品


在线购物系统
localhost:5173
暂无图片
测试商品
价格：￥12.5
描述：用于演示买家端首页在售商品展示
我要购买
C:\WINDOWS\system32\cmd.
VITE v5.4.21 ready in 4757 ms
➜ Local: http://localhost:5173/
➜ Network: use --host to expose
➜ press h + enter to show help
18:46:40 [vite] http proxy error: /api/product/on-sale
Error: connect ECONNREFUSED 127.0.0.1:8000
at TCPConnectWrap.afterConnect [as oncomplete] (node:net:1637:16)
Windows PowerShell
2026-10-21T18:46:50.633+08:00 DEBUG 129584 --- [onlineshop-backend] [main] c.o.mapper.SellerMapper.selectCount : ==> Parameters:
2026-10-21T18:46:50.650+08:00 DEBUG 129584 --- [onlineshop-backend] [main] c.o.mapper.SellerMapper.selectCount : <== Total: 1
2026-10-21T18:46:50.652+08:00 DEBUG 129584 --- [onlineshop-backend] [main] c.o.mapper.SellerMapper.selectList : ==> Preparing: SELECT id,username,password,password_version FROM seller
2026-10-21T18:46:50.653+08:00 DEBUG 129584 --- [onlineshop-backend] [main] c.o.mapper.SellerMapper.selectList : ==> Parameters:
2026-10-21T18:46:50.655+08:00 DEBUG 129584 --- [onlineshop-backend] [main] c.o.mapper.SellerMapper.selectList : <== Total: 1
2026-10-21T18:46:56.779+08:00 INFO 129584 --- [onlineshop-backend] [nio-8000-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
2026-10-21T18:46:56.782+08:00 INFO 129584 --- [onlineshop-backend] [nio-8000-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2026-10-21T18:46:56.789+08:00 INFO 129584 --- [onlineshop-backend] [nio-8000-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 7 ms
2026-10-21T18:46:56.914+08:00 DEBUG 129584 --- [onlineshop-backend] [nio-8000-exec-1] c.o.mapper.ProductMapper.selectList : ==> Preparing: SELECT id,name,description,image,price,status,create_time,finish_time FROM product WHERE (status IN (?,?)) ORDER BY id DESC LIMIT 1
2026-10-21T18:46:56.920+08:00 DEBUG 129584 --- [onlineshop-backend] [nio-8000-exec-1] c.o.mapper.ProductMapper.selectList : ==> Parameters: ON_SALE(String), FROZEN(String)
2026-10-21T18:46:56.929+08:00 DEBUG 129584 --- [onlineshop-backend] [nio-8000-exec-1] c.o.mapper.ProductMapper.selectList : <== Total: 1

2.购买表单


在线购物系统
localhost:5173/buy
提交购买意向
* 姓名
请填写姓名
请填写姓名
* 电话
请填写 11 位手机号
提交

3.表单校验



4.提交成功 + 口令码 + 位次



5.口令查询结果



6.多人提交意向



7.冻结（sql直接修改演示-UPDATE onlineshop.product SET status='FROZEN' WHERE id=1;）



技术栈与结构

- `Vue 3` + `Vite` + `Element Plus`，组合式 API，统一用 `<script setup>`。
- 路由 `src/router/index.js` 使用 `createWebHistory`，买家四个页面都走懒加载，首屏会按需拆包。
- 状态/请求没有引入重方案：买家免登录，所以只用了 `ref/reactive/computed` 和 Axios；Pinia 主要是卖家端在用。
- Vite 在 `vite.config.js` 配置了代理：`/api` → `http://127.0.0.1:8000`，开发环境不用处理跨域。

四个页面的实现

- `Home.vue`

- `onMounted` 请求 `GET /api/product/on-sale`，后端现在会在售或冻结商品都返回。
- 用 `isFrozen = computed(() => product.value?.status === 'FROZEN')` 判断是否交易中。
- 冻结时显示 `el-alert`“商品交易中”，同时禁用“我要购买”，但商品信息仍正常渲染。
- 图片用 `@error` 做失败兜底，`image` 为空或加载失败显示“暂无图片”占位。

- `BuyForm.vue`

  - `el-form` 的 `rules`：姓名必填；电话必填且 `^1\d{10}$`，和后端 `SubmitIntentRequest` 的校验一致。
  - 提交时先 `validate().catch(() => false)`，通过后才发请求，避免无效请求。
  - 调 `POST /api/buyer/intent`，成功后 `router.push` 到 `/done/:code`，并把 `position`、`createTime` 通过 query 带过去。
  - 失败提示交给 Axios 拦截器统一处理，页面只负责 `loading` 状态。
- `Done.vue`

  - 从 `route.params.code` 读口令码，从 `route.query.position` 读位次。
  - 复制口令码用 `navigator.clipboard.writeText`，失败时降级提示手动复制。
  - “去查询”会把口令码通过 query 带到查询页，自动回填。
- `Query.vue`

  - 输入框从 `route.query.code` 自动预填。
  - 调 `GET /api/buyer/intent/{code}`，用 `el-descriptions` 展示位次、状态、商品名、姓名、打码电话、提交时间。
  - 查询失败时清空结果，错误文案由拦截器弹出。

接口封装与错误处理

- 新增 `src/api/buyer.js`，只有两个方法：`submitIntent` 和 `getIntentByCode`。
- Axios 实例在 `src/utils/http.js`：`baseURL = '/api'`、超时 8 秒、`withCredentials`。
- 响应拦截器：`code === 200` 时直接返回 `res.data.data`，所以页面拿到的是纯业务数据。
- 错误拦截器：后端返回真实 HTTP 状态码，401 跳登录，其余弹出后端 `message`，所以页面不用重复写错误提示。

联调机制

- 后端契约是 `接口契约-买家意向.md`，字段和时间格式都约定好了。
- 提交成功返回：`code / position / createTime`。
- 查询返回：`code / position / status / statusText / name / phone / productName / createTime`。
- 业务错误如“暂无在售商品”“商品交易中”“口令码已失效”都由后端返回，前端直接展示。

部分设计原因

- 电话校验放在前端又放后端：前端给即时反馈，后端保证数据安全，两层规则保持一致。
- 用 `computed` 判断冻结而不是写死字符串：商品状态是后端枚举，前端只认 `FROZEN`，状态扩展时不改页面逻辑。
- 接口封装单独放 `buyer.js`：买家、卖家接口边界清晰，后续加撤销接口时只改这一处。